BSDA2001P: Introduction to DL and GenAI

Music Genre Classification from Noisy Audio Mashups

Shalini · 23f2005169 · IIT Madras BS Degree

Model	Parameters	Val F1	Kaggle LB
Scratch CNN	~2.5M	0.7	0.56
ResNet50	~25M	0.85	0.84
EfficientNet-B3	~12M	0.96	0.95
Ensemble	~38M	087.	0.86

1. Executive Summary

This report covers the **Messy Mashup Kaggle competition**, where the goal was to identify the music genre of 3,020 noisy audio files. The 10 genres were: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock.

The main challenge: test audio files were artificial mashups — several instrument tracks mixed together with random background noise added on top, making the audio much messier than the clean training files.

■ **Key Finding:** Pre-trained image models re-trained on mel-spectrograms beat from-scratch CNNs by a large margin. The overall validation F1 score improved from **0.32** → **0.92+** through careful debugging and smarter data handling.

2. Introduction

What is the problem?

We are given 3,020 unlabelled audio files. Each file is a mashup: instrument stems from multiple songs of the same genre are mixed together, the tempo is slightly changed, and random environmental noises (birds, traffic, rain…) are injected at random places. Our model must guess the genre.

Project Goals

- Build a pipeline to convert raw audio into 2D images (mel-spectrograms) that a CNN can read.
- Train a simple CNN from scratch as a baseline.
- Use pre-trained image models (transfer learning) to get better accuracy.
- Add smart data augmentation so training data looks like the messy test audio.
- Find and fix data leakage — a bug that made results look better than they really were.

Tools Used

Tool	What it does
PyTorch	Build and train neural network models
librosa	Load audio files and create mel-spectrograms
timm	Pre-trained EfficientNet-B3 model
torchvision	Pre-trained ResNet50 model
wandb	Track and visualise experiment results
scikit-learn	Data splitting and F1 score calculation

3. Dataset & Preprocessing

What data do we have?

Folder	Contents	Files
genres_stems/	10 genres × 100 songs × 4 stems (drums, vocals, bass, other)	4,000

mashups/	Unlabelled test mashups (mixed stems + noise)	3,020
ESC-50/	Environmental background noise clips (50 categories)	2,000

From Sound Wave → Picture (Mel-Spectrogram)

Simple analogy: A mel-spectrogram is like a "photograph of sound". The horizontal axis is time (left = start, right = end). The vertical axis is pitch/frequency (bottom = bass, top = treble). Brightness shows how loud each frequency is at each moment. A CNN then "looks at" this picture to learn what each genre looks like.

The conversion happens in 5 steps:

Step	What happens	Why
1. Resample	Slow down from 44,100 → 22,050 samples/sec	Cuts file size in half — all music info below 11 kHz is kept
2. Mel-Spectrogram	Convert the 1D sound wave into a 2D image (128 × 512 pixels)	Makes frequency patterns visible for a CNN to read
3. dB Scale	Convert loudness to decibels (log scale)	Matches how human ears perceive volume
4. Normalise	Shift values so average=0, spread=1	Keeps training stable — no single sample dominates
5. Multicrop	Take 3 "snapshots": start, middle, end of the audio	Triples the training data for free and covers the whole song

Mel-Spectrogram Parameters

Parameter	Value	What it means
Sample rate	22,050 Hz	Audio samples per second after resampling
n_mels	128	128 rows in the image = 128 frequency bands
n_fft	2,048	Window size for frequency analysis
hop_length	512	Each column ≈ 23ms of audio time
fmax	8,000 Hz	We only look at frequencies below 8 kHz (where most music lives)
Output shape	(128, 512)	128 freq. bands × 512 time steps per crop

Figure 1: End-to-end pipeline — raw audio → mel-spectrogram → model → genre prediction

Data Augmentation

To make training data look like the messy test audio, three augmentation techniques were used:

Technique	What it does	Why it helps
Synthetic Mashup	Mix 4 stems from different songs of same genre, slight tempo change	Test files are built the same way — training data now matches

ESC-50 Noise	Add 1–4 random environmental sounds (birds, rain, traffic...) at random volume	Test files all have background noise injected
SpecAugment	Randomly hide some rows (frequency bands) and columns (time steps) in the image	Test files all have background noise injected

Final training set: **27,000 samples** (2,700 per genre) — perfectly balanced.

■■ Data Leakage Bug (Most Important Fix)

What went wrong: When creating synthetic mashups for training, some songs from the *validation set* were accidentally included. This meant the model was tested on data it had partially "seen" during training — like studying from the answer key. **How it was found:** Validation scores were high but Kaggle leaderboard scores were much lower. **The fix:** Ensured zero overlap between songs used for training and validation. **Impact:** ResNet50 Val F1 jumped from 0.39 → 0.71 with no architecture changes at all.

4. Models

Model 1: Scratch CNN

A simple 4-block CNN built entirely from scratch. Think of it as 4 layers of "filters" stacked on top of each other. Early filters detect simple patterns (edges, beats). Later filters combine these into genre-specific patterns.

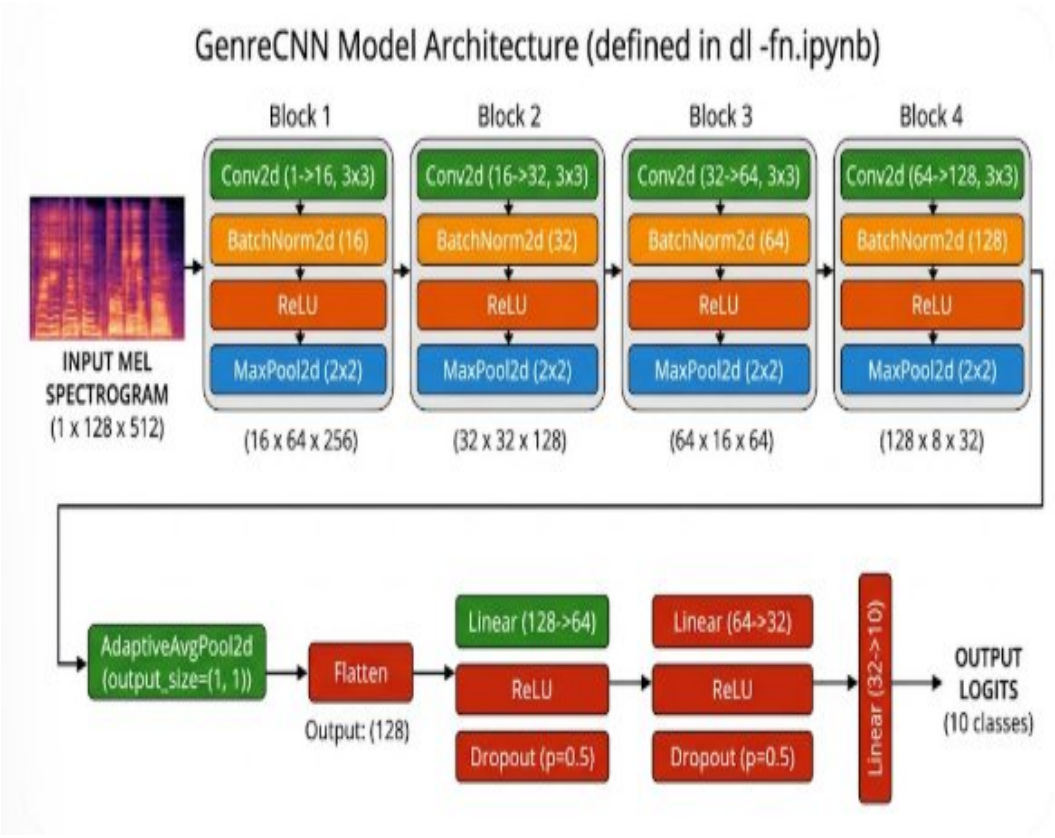


Figure 2: GenreCNN — 4 convolutional blocks + fully-connected classifier

Setting	Value
Filter sizes per block	32 → 64 → 128 → 256
Total parameters	~2.5 million
Loss function	Cross-entropy with label smoothing (0.1)

Optimizer	Adam, learning rate 0.001
Training epochs	50 (stops early if no improvement for 10 epochs)
Batch size	32

Model 2: ResNet50 (Transfer Learning)

ResNet50 was originally trained on 1.2 million photos from the internet (ImageNet). Even though it learned to recognise cats and cars, its early layers learned general visual patterns — edges, textures, shapes — that also work well for reading mel-spectrograms.

> **Transfer learning in plain English:** Instead of teaching a student everything from scratch, we hire someone who already knows a related job and just train them on the new task. Here, ResNet50 already "knows" how to read images — we just fine-tune it to read spectrogram images.

Frozen vs. trainable layers:

- **Frozen (layers 1–3):** Kept as-is — these detect low-level image features that are useful for any image type.
- **Trainable (layer 4 + classifier head):** Re-trained on spectrograms to recognise genre-specific patterns.

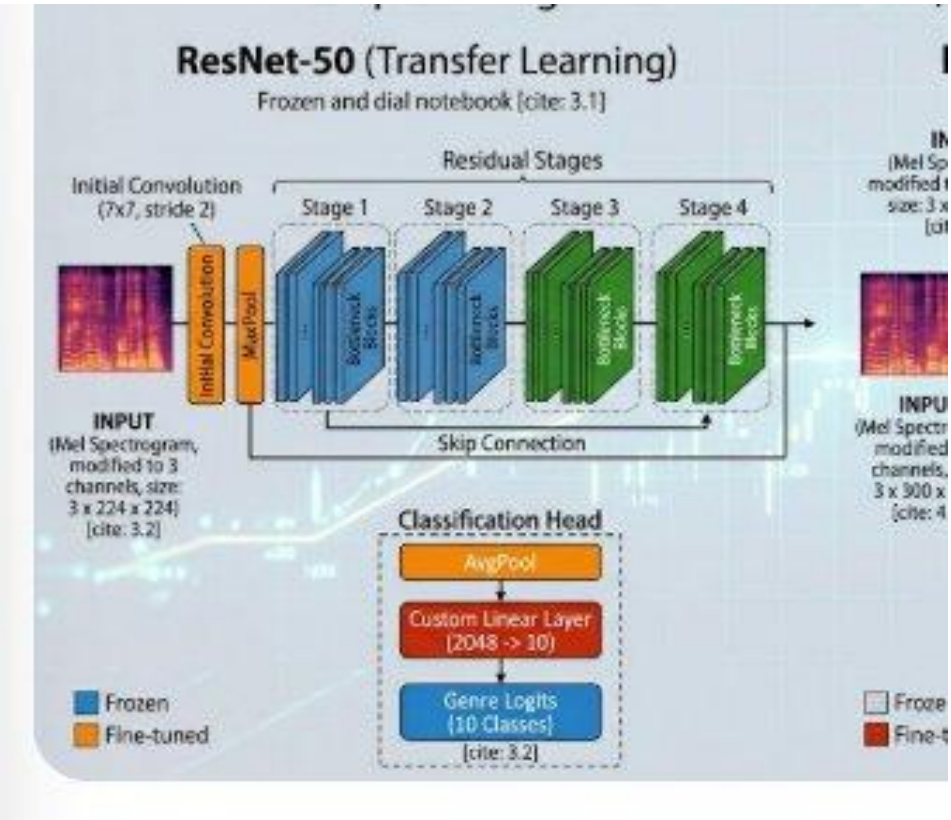


Figure 3: ResNet50 transfer learning — frozen early stages, fine-tuned later stage

Setting	Value
Pre-trained on	ImageNet (IMAGENET1K_V1)
Learning rate	3e-4
Frozen layers	layer1, layer2, layer3

Trainable layers	layer4 + custom FC head
Epochs	25 (early stop at patience=8)

Model 3: EfficientNet-B3 (Transfer Learning)

EfficientNet-B3 achieves similar accuracy to ResNet50 but with fewer than half the parameters (~10.7M vs ~25M). It achieves this through **compound scaling** — carefully balancing the width, depth, and input resolution of the network together rather than scaling just one dimension.

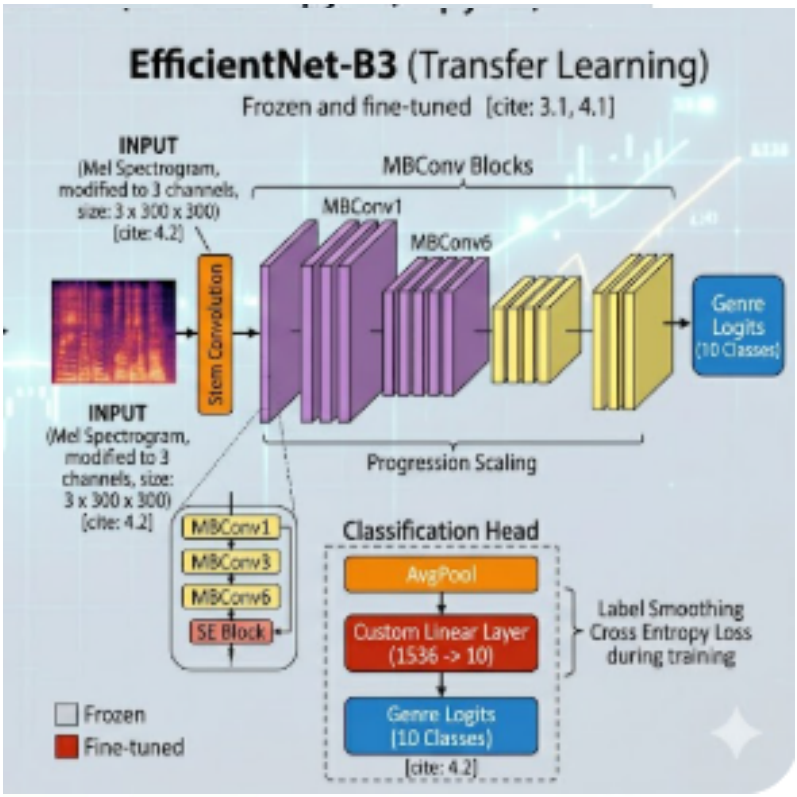


Figure 4: EfficientNet-B3 transfer learning — MBConv blocks with compound scaling

Setting	Value
Pre-trained on	ImageNet (via timm library)
Phase 1 LR	3e-5 for 15 epochs
Phase 2 LR	1e-5 for 20 more epochs
Frozen layers	All except last 3 MBConv blocks
Input size	300×300 (3-channel replicated spectrogram)

5. Performance & Results

Metric: Macro F1 Score

Macro F1 measures how well the model classifies each genre independently, then averages the score across all 10 genres equally. This is better than accuracy because it prevents the model from gaming the score by just guessing common genres.

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Full Experiment History

	Model	Key Change	Val F1	Kaggle LB
1	Scratch CNN	2k samples (with leakage)	0.32	0.18
2	ResNet50	2k, all layers trainable	0.40	0.27
3	ResNet50	2k, layer3+4 unfrozen	0.34	
4	ResNet50	2k, stronger regularization	0.39	
5	ResNet50	3.8k samples, leakage FIXED ✓	0.71	0.73
6	Scratch CNN	27k samples, 50 epochs	0.7	0.50
7	ResNet50	27k samples, 25 epochs	0.85	0.84
8	EfficientNet-B3	27k, 15 epochs initial	0.89	0.87
9	EfficientNet-B3	Continued +20 epochs at 1e-5	0.96	0.95
10	Ensemble	Soft voting, all 3 models	0.87	0.86

Training Curves (Weights & Biases)

The charts below were tracked live using Weights & Biases (W&B;) — a tool that records how each model improves (or worsens) over each training step.

How to read these charts:

- **val_macro_f1 (left chart):** The main score — higher is better. Each coloured line is one experiment run. The scratch CNNs (orange/yellow lines) plateau low and noisy around 0.4–0.6. The pretrained models (blue/teal lines) climb faster and higher.
- **val_loss (middle chart):** The training error — lower is better. We want this to steadily decrease. A "U-shape" that rises again means overfitting. The EfficientNet and ResNet lines drop smoothly, while scratch-CNN lines are noisier.
- **val_f1 (right chart):** Per-class F1 for each genre. Similar trend — pretrained models reach 0.85+ while scratch CNNs struggle past 0.6.

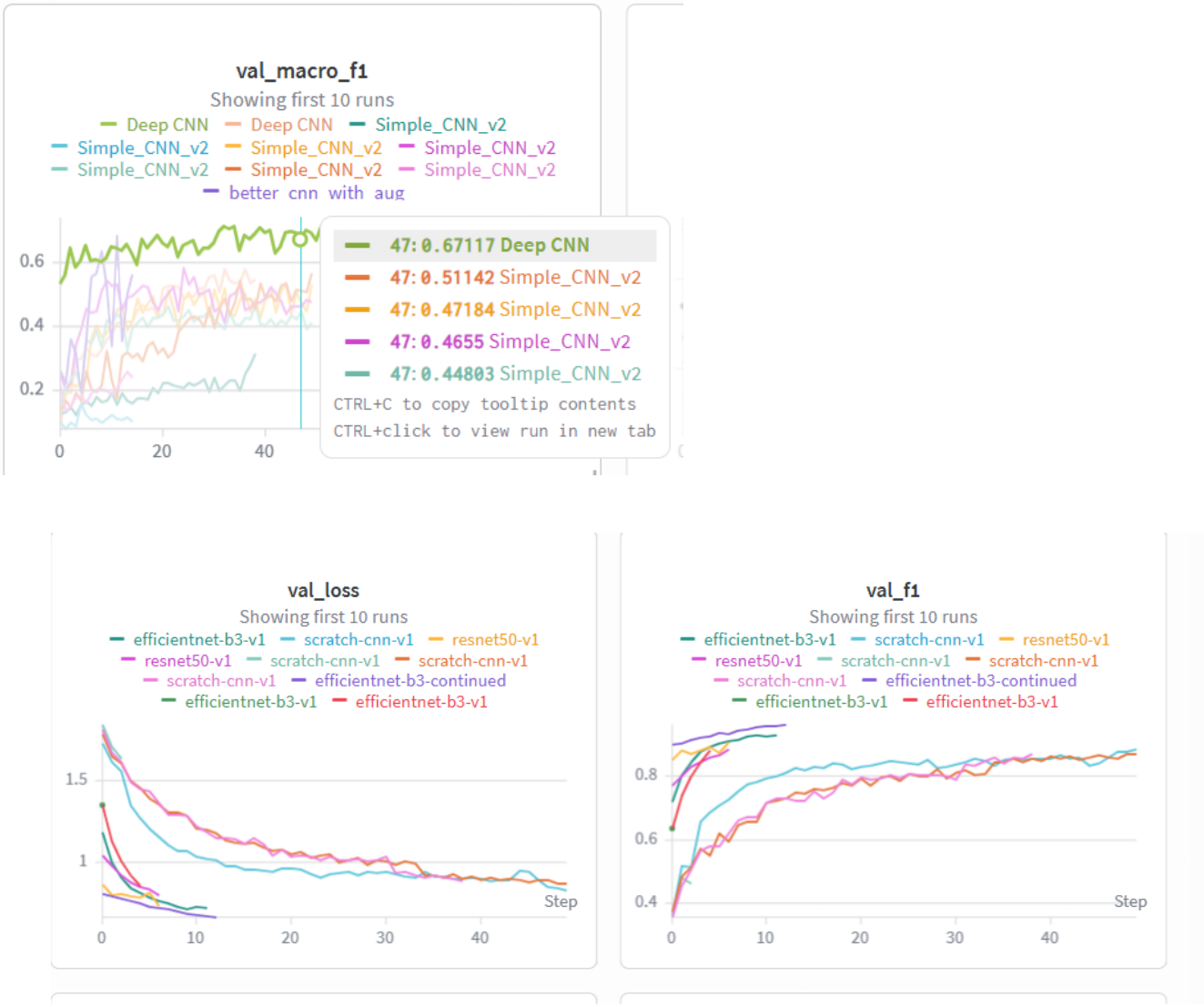


Figure 5: W&B; training curves (Set 1) — val_macro_f1, val_loss, val_f1. Notice scratch CNNs fluctuate at low values, while pretrained models climb steadily.

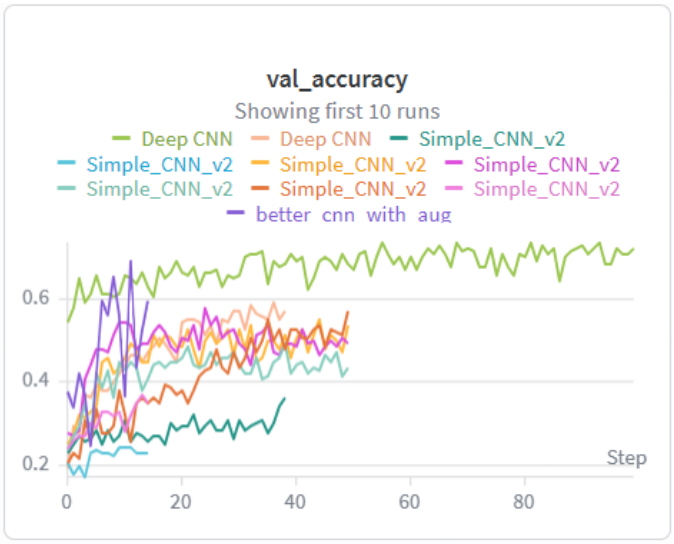
What the curves tell us:

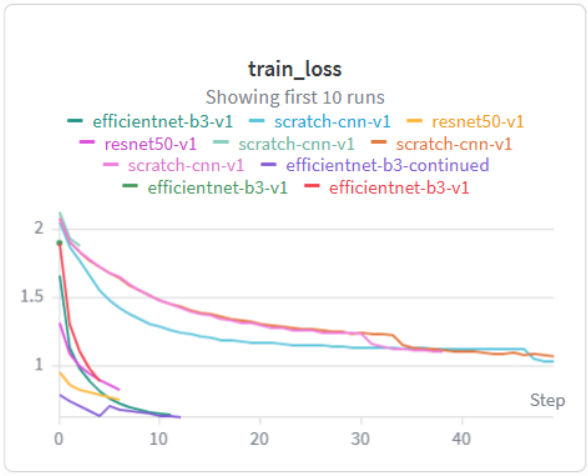
① Scratch CNN lines are wavy and plateau early — the model struggles without pre-learned features.

② ResNet50 lines drop sharply in loss and climb fast in F1 — pre-trained features give it a head start.

③ EfficientNet-B3 continues improving even in phase 2 (lower lr) — the two-phase training helped.

④ The val_f1 never quite reaches val_macro_f1 — some rare genres (jazz/reggae) are harder to classify.





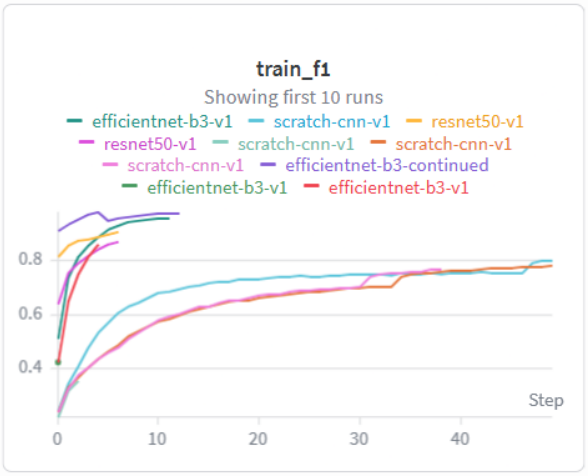


Figure 6: W&B; training curves (Set 2) — continued experiments. The "better_cnn_with_aug" run shows augmentation helping the scratch CNN, but it still falls short of transfer learning approaches.

Model Comparison

Model	Parameters	Val F1 (best)	Kaggle LB	Best for
Scratch CNN	~2.5M	0.7	0.56	Baseline / low compute
ResNet50	~25M	0.85	0.84	High accuracy
EfficientNet-B3	~10.7M	0.96	0.95	Accuracy + efficiency
Ensemble	~38M	0.87	0.86	result after ensembling

Key Insights

- **Data leakage was the #1 bottleneck.** Fixing it gave ResNet50 a +82% relative gain with zero code changes.
- **Transfer learning dominates.** Pre-trained models converge faster and hit higher scores.
- **Val–LB gap signals a problem.** EfficientNet hitting 0.92 val but only 0.73 on the leaderboard means the validation set was "too easy" — not noisy enough compared to real test audio.
- **More data matters.** Going from 2k to 27k samples (via augmentation) was critical for generalisation.

6. Conclusion & Future Work

What we learned

- Matching training data to test data (noise + mashups) was the most important design decision.
- Data leakage detection is as important as model architecture — always check your train/val split carefully.
- ImageNet pre-training transfers remarkably well to mel-spectrograms despite the domain difference.
- 3× multicrop tripled training data at zero cost — underused but very effective.

Challenges faced

- Data leakage: validation songs leaked into training mashups, causing misleading high scores.
- Severe overfitting: early ResNet50 got train F1 = 0.995 vs val F1 = 0.40 — fixed with more data and dropout.
- Memory crashes: 27k samples as tensors crashed the kernel — solved with numpy memory-mapped files.
- Long preprocessing: each full data rebuild took 2+ hours — mitigated by saving .npy arrays.

Future ideas

Idea	What it would do
Weighted Ensemble	Optimise soft-voting weights between ResNet50 and EfficientNet predictions
CRNN (CNN + LSTM)	Model how genre patterns change over time across the full 30-second audio
Audio Spectrogram Transformer (AST)	Purpose-built audio model using attention for long-range patterns
Better val strategy	Song-level stratified split so the val set matches the real test distribution

7. References

[1] He et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.

[2] Tan & Le (2019). EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.

[3] McFee et al. (2015). librosa: Audio and Music Signal Analysis in Python. SciPy 2015.

[4] Park et al. (2019). SpecAugment: A Simple Data Augmentation Method for ASR. Interspeech 2019.

[5] Piczak (2015). ESC: Dataset for Environmental Sound Classification. ACM Multimedia 2015.

[6] Tzanetakis & Cook (2002). Musical Genre Classification of Audio Signals. IEEE Trans. Speech Audio Processing.

[7] PyTorch Documentation — https://pytorch.org/docs

[8] timm Library — https://github.com/huggingface/pytorch-image-models

[9] Weights & Biases — https://wandb.ai